

Software Engineering Fundamentals

A Complete Developer's Guide to
System Design · Architectural Principles
Edge Cases · Constraints

With Real-World Examples in Web & App Development
Version 1.0 · 2025

Table of Contents

Chapter 1 — System Design

- 1.1 What Is System Design?
- 1.2 Core Components
- 1.3 Scalability Patterns
- 1.4 Database Design
- 1.5 Caching Strategies
- 1.6 API Design
- 1.7 Real-World Example: URL Shortener
- 1.8 Real-World Example: Social Media Feed

Chapter 2 — Architectural Principles

- 2.1 SOLID Principles
- 2.2 DRY, KISS, YAGNI
- 2.3 Clean Architecture
- 2.4 Microservices vs Monolith
- 2.5 Event-Driven Architecture
- 2.6 Domain-Driven Design (DDD)
- 2.7 Design Patterns Overview

Chapter 3 — Edge Cases

- 3.1 What Are Edge Cases?
- 3.2 Input Validation Edge Cases
- 3.3 Concurrency & Race Conditions
- 3.4 Network & API Edge Cases
- 3.5 Database Edge Cases
- 3.6 Security Edge Cases
- 3.7 Mobile & Web-Specific Edge Cases
- 3.8 Testing Edge Cases

Chapter 4 — Constraints

- 4.1 Types of Constraints
- 4.2 Performance Constraints
- 4.3 Scalability Constraints
- 4.4 Business & Time Constraints
- 4.5 Security & Compliance Constraints
- 4.6 Managing Trade-offs
- 4.7 Constraint-Driven Design Decisions

System Design

System design is the process of defining the architecture, components, modules, interfaces, and data flow for a system to satisfy specified requirements. It bridges business needs and technical implementation — the blueprint before you write a single line of code.

1.1 What Is System Design?

System design encompasses both High-Level Design (HLD) — how components connect and communicate — and Low-Level Design (LLD) — the internal logic, class structures, and database schemas. Every production web application or mobile app is a result of explicit or implicit design decisions.

- **HLD (High-Level Design):** Architecture diagrams, service boundaries, data flow, technology choices.
- **LLD (Low-Level Design):** Class diagrams, API contracts, database schemas, algorithm choices.
- **Functional Requirements:** What the system must DO (e.g., users can upload photos).
- **Non-Functional Requirements:** How well it must do it (latency < 200ms, 99.9% uptime).

■ Rule of thumb: Always clarify requirements before designing. Ask: How many users? What's the read/write ratio? What's acceptable latency? What data needs to be stored and for how long?

1.2 Core Components of System Design

Client Layer

The client is any device that sends requests — browser, iOS/Android app, desktop app, or even another server. Clients communicate with servers over HTTP/HTTPS, WebSockets, or gRPC depending on the use case.

Load Balancer

A load balancer distributes incoming traffic across multiple server instances. It prevents any single server from becoming a bottleneck and provides fault tolerance if one server goes down.

- **Round Robin:** Requests distributed sequentially across servers.
- **Least Connections:** Traffic sent to server with fewest active connections.
- **IP Hash:** Same client always reaches same server (session persistence).

Web/App Servers

Application servers handle business logic. In modern architectures, these are often stateless microservices — each request carries all the context it needs (via JWT tokens or session IDs), so any server can handle any request.

Databases

Persistence layer. Choice between SQL (PostgreSQL, MySQL) and NoSQL (MongoDB, Cassandra, Redis) depends on data structure, consistency requirements, and query patterns.

Cache

Caches store frequently-accessed data in memory (Redis, Memcached) to reduce database load and improve response times dramatically — from 10ms DB query to 0.1ms cache hit.

Message Queue

Queues (Kafka, RabbitMQ, SQS) decouple services and handle async tasks — email sending, video transcoding, notification delivery — so heavy operations don't block the user response.

CDN (Content Delivery Network)

CDNs serve static assets (images, JS, CSS) from edge servers geographically close to users, dramatically reducing latency for global users. Example: Cloudflare, AWS CloudFront.

Component	Primary Role	Examples	When to Use
Load Balancer	Distribute traffic	Nginx, AWS ALB, HAProxy	Multiple server instances
Cache	Speed up reads	Redis, Memcached	Repeated read-heavy queries
Message Queue	Async processing	Kafka, RabbitMQ, SQS	Decoupling, background jobs
CDN	Static asset delivery	Cloudflare, CloudFront	Global users, media-heavy apps
Database (SQL)	Structured data	PostgreSQL, MySQL	Relational, ACID transactions
Database (NoSQL)	Flexible data	MongoDB, DynamoDB	Unstructured, high scale reads

1.3 Scalability Patterns

Scalability is the ability of a system to handle increased load gracefully. There are two primary scaling strategies:

- **Vertical Scaling (Scale Up):** Add more CPU/RAM to existing servers. Simple but has hardware limits and single point of failure.

- **Horizontal Scaling (Scale Out):** Add more servers. Industry standard for web apps. Requires stateless architecture.

Database Scaling Patterns

- **Read Replicas:** Route all read queries to replica databases; writes go to primary. Works well when reads vastly outnumber writes (social media feeds).
- **Sharding:** Split database into partitions (shards) by key (e.g., `user_id % N`). Each shard handles a subset of data. Enables massive horizontal scale.
- **Connection Pooling:** Maintain a pool of DB connections (PgBouncer) to avoid connection overhead on each request.

■ Python — Consistent Hashing Ring (Database Sharding)

```
# Example: Consistent Hashing for Sharding
import hashlib
class ConsistentHashRing:
    def __init__(self, nodes, replicas=3):
        self.ring = {}
        self.sorted_keys = []
        for node in nodes:
            for i in range(replicas):
                key = self._hash(f'{node}:{i}')
                self.ring[key] = node
        self.sorted_keys.append(key)
        self.sorted_keys.sort()
    def _hash(self, value):
        return int(hashlib.md5(value.encode()).hexdigest(), 16)
    def get_node(self, key):
        h = self._hash(key)
        for ring_key in self.sorted_keys:
            if h <= ring_key:
                return self.ring[ring_key]
        return self.ring[self.sorted_keys[0]] # wrap around
ring = ConsistentHashRing(['db-1', 'db-2', 'db-3'])
print(ring.get_node('user_12345')) # -> 'db-2'
```

1.4 Database Design

Good database design is foundational. Poorly designed schemas cause performance problems that are extremely hard to fix in production.

Normalization vs Denormalization

- **Normalization:** Eliminate redundancy. Store data once. Great for write-heavy apps. (3NF is the common target).
- **Denormalization:** Duplicate data for read performance. Store aggregated counts, embed related data. Common in NoSQL and high-scale SQL systems.

Indexing Strategy

Indexes speed up reads but slow down writes and consume storage. Index columns used in WHERE, JOIN, ORDER BY clauses.

■ SQL — Indexing Strategy

```
-- Good: Compound index for common query pattern
CREATE INDEX idx_posts_user_created ON posts(user_id, created_at DESC);
-- Query that uses the index efficiently
SELECT * FROM posts WHERE user_id = 42 ORDER BY created_at DESC LIMIT 20;
-- Bad: No index, full table scan on 50M rows
SELECT * FROM posts WHERE LOWER(title) LIKE '%search%';
```

1.5 Caching Strategies

- **Cache-Aside (Lazy Loading):** App checks cache first; on miss, reads from DB and populates cache. Most common pattern.
- **Write-Through:** Write to cache and DB simultaneously. Cache always fresh but higher write latency.
- **Write-Back (Write-Behind):** Write to cache, async flush to DB. Fastest writes but risk of data loss.
- **Read-Through:** Cache sits in front of DB; it fetches from DB on miss automatically.

■ JavaScript (Node.js) — Cache-Aside Pattern

```
// JavaScript — Cache-Aside Pattern (Node.js + Redis)
const redis = require('ioredis'); const client = new redis();
async function getUserProfile(userId) {
  const cacheKey = `user:${userId}`; // 1. Check cache first
  const cached = await client.get(cacheKey);
  if (cached) { return JSON.parse(cached); } // Cache HIT
  // 2. Cache MISS — fetch from DB
  const user = await db.query( 'SELECT * FROM users WHERE id = $1', [userId] );
  // 3. Populate cache with TTL (1 hour)
  await client.setex(cacheKey, 3600, JSON.stringify(user));
  return user; }
```

1.6 API Design

APIs are contracts between systems. Good API design makes services composable, maintainable, and developer-friendly.

REST vs GraphQL vs gRPC

Protocol	Best For	Pros	Cons
REST	Public APIs, CRUD apps	Simple, widely understood, HTTP native	Over/under-fetching data
GraphQL	Complex UIs, mobile apps	Precise data fetching, single endpoint, strongly typed	Complex caching, learning curve
gRPC	Internal microservices	Fast (binary), streaming, strongly typed via Protobuf	Not browser-native, harder to debug
WebSocket	Real-time features	Bi-directional, low latency	Stateful, complex scaling

■ Node.js — RESTful API Design

```
# REST API — Express.js with proper conventions
# GET /api/v1/users - list users
# POST /api/v1/users - create user
# GET /api/v1/users/:id - get user
# PUT /api/v1/users/:id - replace user
# PATCH /api/v1/users/:id - update user fields
# DELETE /api/v1/users/:id - delete user
app.get('/api/v1/users/:id', async (req, res) => {
  try {
    const user = await UserService.findById(req.params.id);
    if (!user) return res.status(404).json({ error: 'User not found' });
    res.json({ data: user, timestamp: new Date().toISOString() });
  } catch (err) {
    res.status(500).json({ error: 'Internal server error' });
  }
});
```

1.7 Real-World Example: URL Shortener (bit.ly)

Designing a URL shortener is a classic system design problem that touches on hashing, databases, caching, and scalability. Let's design a system handling 100M URLs and 10B redirects/month.

Requirements

- **Functional:** Given a long URL, return a 7-char short code. Redirect short URL to original.
- **Non-Functional:** 99.99% uptime, redirect latency < 10ms (P99), 100M new URLs/month.
- **Estimated scale:** ~40 writes/sec, ~4,000 redirects/sec peak.

Design Decisions

- **Encoding:** Use Base62 (a-z, A-Z, 0-9) for short codes. 7 characters = $62^7 = 3.5$ trillion unique URLs.
- **Database:** Simple key-value — Redis for active URLs (hot cache), PostgreSQL for persistent storage.
- **ID Generation:** Auto-increment DB ID → encode to Base62. Or use distributed ID (Snowflake ID).
- **Redirect flow:** Client → CDN edge → Cache hit → 301/302 redirect. No app server needed for hot URLs.

■ Python — URL Shortener Base62 Encoding

```
# Python - Base62 Encoder for URL Shortener BASE62 =
'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789'
def encode(num: int) -> str: """Encode a numeric ID to Base62 short code"""
    if num == 0: return BASE62[0]
    result = []
    while num:
        result.append(BASE62[num % 62])
        num //= 62
    return ''.join(reversed(result))
def decode(s: str) -> int: """Decode Base62 short code to numeric ID"""
    result = 0
    for char in s:
        result = result * 62 + BASE62.index(char)
    return result
# DB ID 10000000 -> 'h8G' (much shorter than full ID)
print(encode(10000000)) # -> 'h8G'
print(decode('h8G')) # -> 10000000
```

1.8 Real-World Example: Social Media Feed (Instagram/Twitter)

Designing a feed system is one of the hardest challenges in web engineering. It must be personalized, fast, and handle both celebrity users (millions of followers) and regular users uniformly.

Fan-out Strategies

- **Fan-out on Write (Push):** When user posts, immediately write to all followers' feed caches. Fast reads, expensive writes for celebrities.
- **Fan-out on Read (Pull):** On feed request, fetch posts from all followed users and merge. Fresh data but slow reads.
- **Hybrid:** Push for regular users, pull for celebrities (>1M followers). Used by Twitter/X.

- Instagram uses a hybrid approach: pre-compute feeds for users with normal follower counts, but fetch celebrity posts at read time and merge them in. This balances write cost vs read latency.

Architectural Principles

Architectural principles are the timeless rules that guide how we structure code and systems. They transcend languages and frameworks — a codebase following SOLID in Python looks as clean as one following it in TypeScript.

2.1 SOLID Principles

SOLID is an acronym for five principles introduced by Robert C. Martin (Uncle Bob) that form the foundation of maintainable object-oriented design.

S — Single Responsibility Principle (SRP)

A class (or module/function) should have only one reason to change. It should do one thing and do it well.

■ TypeScript — SRP: Single Responsibility Principle

```
// BAD: UserService does too many things
class UserService {
  createUser(data) { /* DB logic */ }
  sendWelcomeEmail(user) { /* email logic */ }
  generateReport(users) { /* reporting logic */ }
  hashPassword(pwd) { /* crypto logic */ } }
// GOOD: Each class has one responsibility
class UserRepository {
  createUser(data) { /* DB */ } }
class EmailService {
  sendWelcome(user) { /* email */ } }
class ReportGenerator {
  generate(users) { /* report */ } }
class PasswordHasher {
  hash(pwd) { /* crypto */ } }
```

O — Open/Closed Principle (OCP)

Software entities should be open for extension but closed for modification. Add new behavior by adding new code, not changing existing code.

■ TypeScript — OCP: Open/Closed Principle

```
// BAD: Must modify every time a new payment method is added
function processPayment(method, amount) {
  if (method === 'credit') { /* credit logic */ }
  else if (method === 'paypal') { /* paypal logic */ } }
// Adding crypto requires modifying this function!
// GOOD: Open for extension via interface
interface PaymentProcessor {
  process(amount: number): Promise; }
class CreditCardProcessor implements PaymentProcessor { ... }
class PayPalProcessor implements PaymentProcessor { ... }
class CryptoProcessor implements PaymentProcessor { ... }
// Just add new class
```

L — Liskov Substitution Principle (LSP)

Objects of a subclass should be substitutable for objects of their superclass without breaking the program. Subclasses must honor the contract of their parent.

■ Python — LSP: Liskov Substitution Principle

```
# BAD: Square breaks Rectangle's contract
class Rectangle:
    def set_width(self, w):
        self.width = w
    def set_height(self, h):
        self.height = h
    def area(self):
        return self.width * self.height
class Square(Rectangle):
    def set_width(self, w):
        # Breaks LSP!
        self.width = self.height = w
# Side effect! # GOOD: Use composition over inheritance
class Shape:
    def area(self):
        raise NotImplementedError
class Rectangle(Shape):
    def __init__(self, w, h):
        self.w, self.h = w, h
    def area(self):
        return self.w * self.h
class Square(Shape):
    def __init__(self, side):
        self.side = side
    def area(self):
        return self.side ** 2
```

I — Interface Segregation Principle (ISP)

Clients should not be forced to depend on methods they do not use. Split large interfaces into smaller, more specific ones.

D — Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules. Both should depend on abstractions. This is the basis of Dependency Injection — a cornerstone of testable code.

■ TypeScript — DIP: Dependency Inversion with Dependency Injection

```
// GOOD: High-level OrderService depends on abstraction, not concrete DB interface
interface OrderRepository {
    save(order: Order): Promise;
    findById(id: string): Promise;
}
class OrderService {
    constructor(private repo: OrderRepository) {} // DI
    async placeOrder(items: Item[]) {
        const order = new Order(items);
        await this.repo.save(order);
        // Works with Postgres, MongoDB, or mock!
    }
} // In tests: inject a MockOrderRepository // In production: inject PostgresOrderRepository
```

2.2 DRY, KISS, and YAGNI

- **DRY (Don't Repeat Yourself):** Every piece of knowledge must have a single, authoritative representation. Duplication leads to inconsistency when one copy is updated but others aren't.
- **KISS (Keep It Simple, Stupid):** Prefer the simplest solution that works. Complexity is the enemy of maintainability. Don't build a distributed system when a monolith suffices.
- **YAGNI (You Aren't Gonna Need It):** Don't implement features until they're actually needed. Speculative generality adds code, bugs, and cognitive load with no current value.

■■■ A common antipattern for junior developers: building a fully generic, plugin-based, multi-tenant, infinitely configurable system for an app with 3 users. YAGNI. Build what you need today. Refactor when requirements evolve.

2.3 Clean Architecture

Clean Architecture (Robert C. Martin) organizes code into concentric layers where dependencies always point inward. Inner layers know nothing about outer layers.

- **Entities (Core):** Pure business objects and rules. No framework dependencies. Reusable anywhere.
- **Use Cases (Application):** Application-specific business rules. Orchestrate entities.

- **Interface Adapters:** Controllers, presenters, gateways. Convert data between use cases and external formats.
 - **Frameworks & Drivers (Outer):** Web framework, database, UI, external APIs. Replaceable.
- The key test: Can you swap your web framework (Express → Fastify) or database (Postgres → MongoDB) without touching your business logic? If yes, your architecture is clean.

■ Python — Clean Architecture: Entity → Use Case → Controller

```
# Clean Architecture in a Python web app # ENTITY (innermost) - pure business logic,
no imports from outer layers
class Order:
    def __init__(self, items):
        self.items = items
        self.status = 'pending'
    def total(self):
        return sum(i.price for i in self.items)
    def confirm(self):
        if not self.items:
            raise ValueError('Cannot confirm empty order')
        self.status = 'confirmed'
# USE CASE - application logic, depends on entities + abstractions
class PlaceOrderUseCase:
    def __init__(self, order_repo, payment_gateway, notifier):
        self.repo = order_repo # abstraction
        self.payment = payment_gateway # abstraction
        self.notifier = notifier # abstraction
    def execute(self, user_id, items):
        order = Order(items)
        order.confirm()
        self.payment.charge(user_id, order.total())
        self.repo.save(order)
        self.notifier.send_confirmation(user_id, order)
        return order
# CONTROLLER (adapter) - depends on use cases, knows about HTTP
class OrderController:
    def __init__(self, use_case):
        self.use_case = use_case
    def post_order(self, request):
        # HTTP detail stays here
        items = [Item(**i) for i in request.json['items']]
        order = self.use_case.execute(request.user_id, items)
        return {'order_id': order.id, 'total': order.total()}
```

2.4 Microservices vs Monolith

Aspect	Monolith	Microservices
Deployment	Deploy entire app as one unit	Deploy services independently
Scaling	Scale everything together	Scale individual services
Complexity	Low initially, grows over time	High operational complexity from day 1
Team size	Works well with small teams	Suited for large teams / org boundaries
Data	Single shared database	Each service owns its data store
Failure	One failure can crash all	Failure isolated to one service
When to use	Early stage, small team, low scale	Large scale, multiple teams, Netflix/Uber-like

■ Start with a monolith. Extract microservices only when you have a clear seam in your domain, separate team ownership, and proven scaling need. Premature microservices is one of the most expensive architectural mistakes in modern software development.

2.5 Event-Driven Architecture

In event-driven architecture (EDA), components communicate by producing and consuming events via a message broker. Services are decoupled — the producer doesn't know who consumes its events.

■ Node.js — Event-Driven Architecture with Kafka

```
// Node.js — Event-Driven with Kafka (simplified) // Producer: Order Service await
producer.send({ topic: 'order.placed', messages: [{ key: order.id, value:
JSON.stringify({ orderId: order.id, userId: order.userId, total: order.total,
timestamp: Date.now() }) }] }); // Consumer 1: Inventory Service
consumer.subscribe({ topic: 'order.placed' }); // Reduces stock quantities //
Consumer 2: Email Service consumer.subscribe({ topic: 'order.placed' }); // Sends
confirmation email // Consumer 3: Analytics Service consumer.subscribe({ topic:
'order.placed' }); // Records revenue metrics
```

2.6 Domain-Driven Design (DDD)

DDD (Eric Evans) aligns software structure with business domains. It introduces a shared language between developers and domain experts.

- **Bounded Context:** Explicit boundary within which a domain model applies. 'Order' in Shipping context vs 'Order' in Billing context can be different models.
- **Aggregate:** Cluster of objects treated as a unit for data changes. Has one root entity (Aggregate Root).
- **Value Object:** Immutable object defined by its attributes, not identity. E.g., Money(100, 'USD').
- **Domain Event:** A record of something that happened in the domain. E.g., OrderPlaced, PaymentFailed.
- **Repository:** Abstraction for retrieving/persisting aggregates.

2.7 Key Design Patterns in Web/App Development

Pattern	Category	Problem Solved	Example Use
Singleton	Creational	One instance globally	DB connection pool, Config
Factory	Creational	Create objects without exposing logic	Payment processor factory
Observer	Behavioral	Notify dependents of state change	Event system, React hooks
Strategy	Behavioral	Switch algorithms at runtime	Sorting, payment methods
Decorator	Structural	Add behavior without subclassing	Middleware, logging, caching

Pattern	Category	Problem Solved	Example Use
Repository	Architectural	Abstract data access	Data layer in clean architecture
CQRS	Architectural	Separate read/write models	High-scale apps, event sourcing

Edge Cases

Edge cases are inputs or situations that occur at the extreme boundaries of what a system is designed to handle. They're the difference between code that works in demos and code that works in production with real, unpredictable users.

3.1 What Are Edge Cases?

An edge case is any scenario that falls outside the 'happy path' — the normal expected flow. They arise from boundary values, unexpected inputs, concurrent operations, network failures, and the endless creativity of users doing things you never anticipated.

- **Boundary Values:** Minimum, maximum, zero, empty, null, very large numbers.
- **Unexpected Inputs:** SQL injection, XSS payloads, Unicode emoji, extremely long strings.
- **Timing Issues:** Race conditions, timeout, order of operations.
- **Environment Issues:** Network failure mid-request, disk full, memory exhausted.
- **User Behavior:** Double-clicking submit, navigating away mid-upload, refreshing payment page.

3.2 Input Validation Edge Cases

String Fields

- Empty string vs null vs undefined vs whitespace-only string — are these the same?
- String exceeding max length (username = 10,000 chars could break UI or DB column).
- Unicode: emoji (4-byte chars), RTL Arabic/Hebrew text, null bytes (\x00 in strings).
- HTML/script tags in user content — stored XSS if not sanitized.

■ JavaScript — String Input Validation with Edge Cases

```
// JavaScript — Comprehensive string validation function validateUsername(input) {  
// Edge case: null/undefined if (input == null) throw new Error('Username is  
required'); // Edge case: not a string (e.g., array or number passed) if (typeof  
input !== 'string') throw new Error('Username must be a string'); // Edge case:  
whitespace-only const trimmed = input.trim(); if (trimmed.length === 0) throw new  
Error('Username cannot be blank'); // Edge case: too short / too long if  
(trimmed.length < 3) throw new Error('Username too short (min 3)'); if  
(trimmed.length > 30) throw new Error('Username too long (max 30)'); // Edge case:  
SQL injection / special chars if (!/^[a-zA-Z0-9_.-]+$/.test(trimmed)) { throw new  
Error('Username contains invalid characters'); } return trimmed; }
```

Numeric Fields

- Integer overflow: what if quantity = 2,147,483,648 (exceeds 32-bit int)?
- Floating point precision: $0.1 + 0.2 = 0.30000000000000004$ in JavaScript.
- Negative values: price = -50, quantity = -1, age = -5.
- Zero division: calculate average when list is empty.
- Very large numbers: user enters 999999999999 for a quantity field.

■ Python — Safe Numeric Handling for Financial Calculations

```
# Python – Safe numeric operations for financial data from decimal
import Decimal, InvalidOperation
def calculate_total(price_str, quantity_str):
    # Edge case: non-numeric input
    try:
        price = Decimal(str(price_str))
    except:
        # Use Decimal for money!
        raise ValueError('Invalid numeric input')
    quantity = Decimal(str(quantity_str))
    # Edge case: negative values
    if price < 0:
        raise ValueError('Price cannot be negative')
    if quantity <= 0:
        raise ValueError('Quantity must be positive')
    # Edge case: unreasonably large values
    MAX_QUANTITY = Decimal('10000')
    if quantity > MAX_QUANTITY:
        raise ValueError(f'Quantity exceeds maximum ({MAX_QUANTITY})')
    # 0.1 + 0.2 precision issue avoided with Decimal
    return price * quantity
# Exact decimal arithmetic
```

Date/Time Edge Cases

- Timezones: user in Dubai creates an event 'midnight Jan 1' — midnight in which timezone?
- Daylight Saving Time: 2:30 AM on the DST transition day may not exist or may exist twice.
- Leap years: Feb 29 is only valid in leap years.
- Date ordering: start_date > end_date — always validate.
- Epoch time: Unix timestamps stored as 32-bit int will overflow in 2038 (Y2K38).

■ Best practice: Always store datetimes in UTC in your database. Convert to user's local timezone only for display. Never store 'local time without timezone info' — it becomes ambiguous within months.

3.3 Concurrency & Race Conditions

Race conditions occur when multiple processes or threads access shared state simultaneously and the outcome depends on the order of execution. They're some of the hardest bugs to reproduce and fix because they're timing-dependent.

Classic Race Condition: Double Spend

■ SQL — Preventing Race Conditions with Locking

```
-- BAD: Race condition – two requests can both see balance = 100 -- and both deduct 80, leaving balance at -60
SELECT balance FROM accounts WHERE id = 1; -- Returns 100
-- (another request runs the same SELECT here!)
UPDATE accounts SET balance = balance - 80 WHERE id = 1;
-- GOOD: Use SELECT FOR UPDATE to acquire row-level lock
BEGIN;
SELECT balance FROM accounts WHERE id = 1 FOR UPDATE;
-- Now other transactions WAIT here until we COMMIT
UPDATE accounts SET balance = balance - 80 WHERE id = 1 AND balance >= 80;
-- Also check condition atomically
COMMIT;
-- EVEN BETTER: Atomic operation with optimistic locking
UPDATE accounts SET balance = balance - 80, version = version + 1 WHERE id = 1 AND balance >= 80 AND version = 5;
```

Idempotency

Idempotent operations produce the same result whether called once or multiple times. Critical for payment systems, order placement, and any API that can be retried.

■ Node.js — Idempotency Key Pattern (Payments)

```
// Node.js – Idempotency key pattern for payments app.post('/api/payments', async
(req, res) => { const { amount, idempotencyKey } = req.body; // Check if we've
already processed this request const existing = await
redis.get(`idem:${idempotencyKey}`); if (existing) { return
res.json(JSON.parse(existing)); // Return same response! } // Process payment const
result = await paymentGateway.charge(amount); // Store result for 24 hours await
redis.setex(`idem:${idempotencyKey}`, 86400, JSON.stringify(result));
res.json(result); });
```

3.4 Network & API Edge Cases

- **Timeout:** What if the downstream service takes 30 seconds? Set explicit timeouts (e.g., 5s) and return a meaningful error.
- **Partial failure:** You write to DB successfully but the cache update fails — system is now inconsistent.
- **Retry storms:** If 10,000 clients all retry at the same moment after a failure, you get a thundering herd. Use exponential backoff with jitter.
- **Response payload too large:** API returns 50MB of data — client OOMs or connection times out. Always paginate.
- **Circular dependency:** Service A calls B, B calls A during the same request — deadlock.

■ Python — Retry with Exponential Backoff and Jitter

```
# Python – Retry with exponential backoff and jitter import random, time, requests
def fetch_with_retry(url, max_retries=4): for attempt in range(max_retries): try:
resp = requests.get(url, timeout=5) # Explicit timeout resp.raise_for_status()
return resp.json() except requests.exceptions.Timeout: if attempt == max_retries -
1: raise # Final attempt, propagate error except requests.exceptions.HTTPError as
e: if e.response.status_code < 500: raise # 4xx errors – don't retry (client error)
# Exponential backoff with jitter wait = (2 ** attempt) + random.uniform(0, 1)
time.sleep(wait) # 1s, 2s, 4s, 8s (approx)
```

3.5 Database Edge Cases

- **NULL propagation:** In SQL, NULL + anything = NULL. SELECT 100 + NULL returns NULL, not 100.
- **Empty result set:** findById returns no rows — always handle the 'not found' case explicitly.
- **Duplicate unique constraint:** Two concurrent inserts with same email — one will throw. Handle gracefully.
- **Transaction rollback:** If step 3 of a 5-step transaction fails, steps 1-2 must be rolled back.

- **N+1 query problem:** Fetching 100 posts and then making 100 separate queries for each post's author.

■ Python — N+1 Query Problem and Eager Loading Solution

```
# Python/SQLAlchemy – Solving the N+1 Problem # BAD: N+1 – 1 query for posts + N
queries for authors posts = session.query(Post).all() # 1 query for post in posts:
author = post.author # N queries! print(f'{post.title} by {author.name}') # GOOD:
Eager loading – 1 query total with JOIN from sqlalchemy.orm import joinedload posts
= session.query(Post)\ .options(joinedload(Post.author))\ .all() # Single JOIN
query for post in posts: print(f'{post.title} by {post.author.name}') # No extra
queries
```

3.6 Security Edge Cases

- **SQL Injection:** Input: ' OR '1'='1 in a login form can bypass auth if queries aren't parameterized.
- **XSS (Cross-Site Scripting):** Stored user content rendered as HTML can execute attacker JavaScript in victims' browsers.
- **IDOR (Insecure Direct Object Reference):** GET /invoices/1234 — can user 456 access user 123's invoice by changing the ID?
- **Mass Assignment:** POST /users with JSON body including isAdmin:true — does your ORM blindly assign all fields?
- **JWT Algorithm Confusion:** Accepting 'alg:none' in JWT headers — allows forging any token.

■ Python — SQL Injection Prevention

```
# Python – Parameterized queries (NEVER string-format SQL) # BAD: SQL Injection
vulnerability username = request.form['username'] query = f"SELECT * FROM users
WHERE username = '{username}'" # Input: '; DROP TABLE users; -- -> disaster! # GOOD:
Always use parameterized queries cursor.execute( 'SELECT * FROM users WHERE
username = %s', (username,) # Parameter, not string concatenation ) # GOOD:
ORM-based (SQLAlchemy) – parameterized automatically user =
session.query(User).filter( User.username == username # ORM handles escaping
).first()
```

3.7 Mobile & Web-Specific Edge Cases

Web-Specific

- Browser back/forward button restoring stale state after form submission.
- User opens the same app in multiple tabs — shared state (auth, cart) must stay in sync.
- JavaScript disabled — does your app degrade gracefully?
- Slow 3G connection — API takes 10 seconds. Show loading state. Handle timeout.
- Cookie/localStorage full — silent write failures if quota exceeded.

Mobile-Specific

- App goes to background mid-operation (phone call received) — operation must resume or fail cleanly.
- Network switches from WiFi to cellular mid-upload — connection drops, need resumable upload.
- Low storage on device — trying to cache a 50MB file on a device with 10MB free.
- Different screen densities (1x, 2x, 3x) — images must be responsive or appear blurry.
- Permissions revoked after app startup — camera/location may be unavailable mid-flow.

3.8 Testing Edge Cases Systematically

The best way to handle edge cases is to discover them systematically during development rather than in production.

- **Boundary Value Analysis:** Test at min, max, min-1, max+1 for any range.
- **Equivalence Partitioning:** Divide inputs into valid/invalid classes; test one from each.
- **Property-Based Testing:** Generate random inputs and verify invariants hold (Hypothesis in Python, fast-check in JS).
- **Mutation Testing:** Introduce deliberate code bugs — if tests don't catch them, coverage is insufficient.
- **Chaos Engineering:** Intentionally kill services, inject latency, corrupt data — does the system recover?

■ Python — Property-Based Testing with Hypothesis

```
# Python — Property-based testing with Hypothesis
from hypothesis import given, strategies as st

# Instead of writing 10 handpicked test cases, # Hypothesis
# generates HUNDREDS of edge cases automatically
@given(st.text(min_size=0, max_size=1000))
# Any string 0-1000 chars
def test_encode_decode_roundtrip(s):
    """Encoding then decoding should return original string"""
    assert decode(encode(s)) == s

@given(st.integers(), st.integers(min_value=1))
def test_division_always_less_than_numerator(a, b):
    """a/b should always be <= a for positive b"""
    result = safe_divide(a, b)
    assert result <= abs(a)
```

Constraints

Constraints are the boundaries within which a system must operate. They shape every architectural decision. Understanding and explicitly stating constraints early prevents costly redesigns later.

4.1 Types of Constraints

Type	Examples	Impact
Performance	Latency < 100ms, Throughput > 10K RPS	Architecture, database, caching choices
Scalability	10M users, 1B events/day	Horizontal scaling, sharding, CDN
Business	Budget \$5K/month infra, Launch in 3 months	Tech stack, team size, feature scope
Regulatory	GDPR, HIPAA, PCI-DSS, SOC2	Data storage, encryption, audit logging
Technical	Must use AWS, existing PostgreSQL, Python only	Integration complexity, vendor lock-in
Availability	99.99% uptime (52 min downtime/year)	Redundancy, failover, disaster recovery
Team	3 engineers, no DevOps specialist	Managed services vs self-hosted, automation

4.2 Performance Constraints

Performance constraints define acceptable thresholds for speed and throughput. They must be specific and measurable — 'fast' is not a constraint, 'P95 latency < 200ms' is.

Latency Percentiles

Always specify latency as percentiles, not averages. An average of 50ms can hide that 5% of requests take 5 seconds — those 5% are real users having terrible experiences.

- **P50 (Median):** 50% of requests complete within this time. The 'typical' experience.
- **P95:** 95% of requests complete within this time. Most users' worst-case experience.
- **P99:** 99% — even your slow outlier tail. Important for SLAs.

- **P99.9:** 1 in 1000 requests. Critical for high-volume services (1M RPS = 1000 slow requests/second).

■ Python — Measuring and Asserting Performance Constraints

```
# Python – Performance profiling and constraint validation
import time, statistics
def measure_performance(func, inputs, iterations=1000):
    latencies = []
    for _ in range(iterations):
        start = time.perf_counter()
        func(inputs)
        latencies.append((time.perf_counter() - start) * 1000) # ms
    latencies.sort()
    return {
        'p50': latencies[int(iterations * 0.50)],
        'p95': latencies[int(iterations * 0.95)],
        'p99': latencies[int(iterations * 0.99)],
        'mean': statistics.mean(latencies)
    }
results = measure_performance(search_users, 'alice')
assert results['p99'] < 200, f"P99 {results['p99']:.1f}ms exceeds 200ms SLA"
```

Back-of-Envelope Calculations

A critical skill in system design is estimating resource requirements before building. These 'napkin math' calculations guide architectural decisions.

■ Python — Back-of-Envelope Calculation for Photo Storage

```
# System Design: Back-of-envelope for photo storage (Instagram-scale)
# Assumptions
daily_active_users = 500_000_000 # 500M DAU
photos_per_user_day = 0.1 # 1 in 10 users posts daily
avg_photo_size_mb = 3 # After compression
read_write_ratio = 100 # 100 reads per write
retention_years = 5 # Write throughput
photos_per_day = daily_active_users * photos_per_user_day # 50M
photos_per_sec = photos_per_day / 86400 # ~580/sec
# Storage
storage_per_year = photos_per_day * avg_photo_size_mb * 365 # ~54 TB/year
total_storage_5y = storage_per_year * retention_years # ~270 TB
# Read throughput
reads_per_sec = photos_per_sec * read_write_ratio # ~58,000 RPS
print(f'Write QPS: {photos_per_sec:.0f}')
print(f'Read QPS: {reads_per_sec:.0f}')
print(f'Storage needed (5yr): {total_storage_5y:.0f} TB')
```

4.3 Scalability Constraints

Scalability constraints define the maximum load the system must handle, which drives decisions about horizontal scaling, database sharding, and stateless design.

Stateless Services

For horizontal scaling to work, application servers must be stateless. All state must live in shared storage (Redis, DB), not in server memory.

■ Node.js — Stateless Auth for Horizontal Scaling

```
// BAD: Stateful server – can't horizontally scale // Session stored in server memory
app.post('/login', (req, res) => { req.session.userId = user.id; // In-memory!
Lost if server restarts
req.session.cart = []; // Different server = empty cart! });
// GOOD: Stateless with JWT – any server can validate
app.post('/login', async (req, res) => { const token = jwt.sign( { userId: user.id, email: user.email }, process.env.JWT_SECRET, { expiresIn: '7d' } );
res.json({ token }); // Client stores token, sends with every request });
// GOOD: Shared session store (Redis) for stateful data
const session = require('express-session');
const RedisStore = require('connect-redis')(session);
app.use(session({ store: new RedisStore({ client: redisClient }), secret: process.env.SESSION_SECRET // Session in shared Redis! }));
```

4.4 Business & Time Constraints

Business constraints are often the most decisive. No engineering solution matters if the product ships 6 months late or costs 10x the budget.

- **Budget constraint:** \$3K/month infra budget eliminates multi-region active-active setups. Forces managed services over self-hosted.
- **Time-to-market:** 3-month deadline for MVP. Use off-the-shelf auth (Auth0, Clerk) instead of building. Use Firebase instead of custom backend.
- **Team constraint:** 2 full-stack devs. Choose a monolith over microservices. Use managed DB over self-hosted.
- **Vendor constraint:** Enterprise client requires on-premise deployment. Container-first architecture (Docker/K8s) becomes mandatory.

■ The 'boring technology' principle (Dan McKinley): Choose established, well-understood tools over cutting-edge ones. Postgres can handle more scale than most startups will ever need. Save your innovation budget for the parts of your product that are actually unique.

4.5 Security & Compliance Constraints

Regulation	Industry	Key Requirements	Technical Implications
GDPR	Any (EU users)	Right to erasure, data portability, consent	Soft deletes, data export API, consent tables
HIPAA	Healthcare (US)	PHI encryption, access logging, BAAs	Encrypt at rest+transit, audit logs, access controls
PCI-DSS	Payment processing	No storing raw card data, network segmentation	Tokenize cards, never store CVV, regular pen tests
SOC 2	SaaS/B2B	Security, availability, confidentiality controls	Audit logging, incident response, access reviews

■ Python — GDPR Right to Erasure Implementation

```
# Python — GDPR Right to Erasure implementation
class GDPRService:
    def erase_user_data(self, user_id: str) -> dict:
        """Handle GDPR Article 17 'Right to be Forgotten'"""
        # 1. Anonymize PII in orders (can't delete — financial records)
        db.execute(""" UPDATE orders SET shipping_address = '[REDACTED]', customer_email = 'deleted@gdpr.invalid' WHERE user_id = %s """, (user_id,))
        # 2. Delete personal profile data
        db.execute('DELETE FROM user_profiles WHERE user_id = %s', (user_id,))
        # 3. Remove from search indexes
        elasticsearch.delete_by_user(user_id)
        # 4. Revoke all active sessions/tokens
        redis.delete_pattern(f'session:{user_id}:*')
        # 5. Audit log the erasure (required by GDPR)
        audit_log.record('GDPR_ERASURE', user_id, datetime.utcnow())
        return {'status': 'erased', 'user_id': user_id}
```

4.6 Managing Trade-offs

Every constraint creates a trade-off. The CAP theorem is the most famous example: in a distributed system, you can guarantee at most two of: Consistency, Availability, Partition tolerance.

Trade-off	Option A	Option B	Choose A when...	Choose B when...
Consistency vs Availability	Strong consistency (ACID)	Eventual consistency (BASE)	Financial data, inventory	Social feeds, analytics
Latency vs Accuracy	Serve cached (stale) data	Query DB for fresh data	Feed items, recommendations	Account balance, stock
Cost vs Performance	Single region, cheaper	Multi-region, expensive	Early stage startup	Enterprise, global users
Build vs Buy	Custom solution	Third-party service	Core differentiator	Auth, email, payments
SQL vs NoSQL	PostgreSQL (relational)	MongoDB/Dynamo DB (document)	Complex queries, joins	Simple lookups, huge scale

4.7 Constraint-Driven Design Decisions

Here's a practical walkthrough: designing a chat app under different constraints leads to fundamentally different architectures.

Scenario A: Small Team, Limited Budget (\$500/month infra)

- Use WebSockets on a single Node.js server with Socket.io.
- Store messages in PostgreSQL (managed: Supabase or Railway).
- Redis Pub/Sub for broadcasting messages between server instances (if you need 2-3 servers).
- No message queue needed — volume doesn't justify Kafka's operational overhead.

Scenario B: 10M Users, WhatsApp-Scale

- Dedicated connection servers (WebSocket tier) — stateful, one per connected client.
- Message routing service — routes messages between connection servers.
- Kafka for message persistence and fan-out to offline users.
- Cassandra for message storage — append-only workload, write-optimized, scales horizontally.
- Push notification service (APNs/FCM) for offline users.
- Message encryption end-to-end (Signal protocol) — a security constraint.

■ The right architecture is the one that fits your actual constraints today, with a clear path to evolve as constraints change. Don't build Scenario B infrastructure for Scenario A — you'll spend 80% of your engineering time on infrastructure instead of product.

SUMMARY

Key Takeaways

- **System Design:** Start with requirements, estimate scale, choose components (load balancer, cache, DB, queue) based on actual needs. Practice designing real systems (URL shortener, social feed, chat app).
- **Architectural Principles:** SOLID, DRY, KISS, YAGNI, Clean Architecture — these are timeless guides. Apply them pragmatically, not dogmatically. The goal is maintainable, testable, understandable code.
- **Edge Cases:** The difference between amateur and professional code. Validate all inputs. Handle null/empty/overflow. Think about concurrency. Test boundaries systematically. Use property-based testing.
- **Constraints:** Make them explicit upfront. Every architectural decision is a trade-off shaped by constraints. Match your architecture to your actual scale, budget, and team — not to the architecture of Netflix.

The best engineers don't just know patterns and principles — they know when to apply them and, equally importantly, when not to. Software engineering is the art of managing complexity under constraints, handling the unexpected, and building systems that can evolve. Keep learning, keep building, and keep questioning your assumptions.

— End of Document —